

Acquisition et Nettoyage des Données

Master 1 Data & IA — 2025-2026

Ahmad ALMAKSOUR — Faculté de Gestion, Économie et Sciences

Plan de la séance

01

Architecture REST

02

Authentification

03

Consommation robuste

04

Pydantic

05

APIs françaises

06

Intro MCP

07

Cas pratique

Pourquoi récupérer des données via des APIs ?

Vous avez manipulé des données déjà téléchargées. Dans la réalité, la majorité des données fraîches, en temps réel ou à la demande, transitent par des APIs.

Pas de fichier à gérer

L'API renvoie toujours la version à jour.
Vous ne stockez que ce que vous demandez, quand vous en avez besoin.

Données fraîches

Un CSV DVF téléchargé en janvier est périmé en mars. Une API renvoie les données du jour.

Automatisation

Un script qui appelle une API toutes les nuits remplace un humain qui télécharge un fichier chaque matin.

Anatomie d'une requête HTTP

Toute communication avec une API REST repose sur une requête HTTP. Comprendre sa structure est le prérequis absolu.

REQUÊTE (Client → Serveur)

```
GET /communes?codeDepartement=59 HTTP/1.1
Host: geo.api.gouv.fr
Accept: application/json
Authorization: Bearer <token> # si requis
User-Agent: Mon-Pipeline/1.0
```

Verbe

GET, POST, PUT, DELETE, PATCH - l'action demandée

Headers

Métadonnées : format accepté, authentification, cache, rate limit

RÉPONSE (Serveur → Client)

```
HTTP/1.1 200 OK
Content-Type: application/json
X-RateLimit-Remaining: 99

[{"nom": "Lille", "code": "59350", "population": 233098}]
```

Code statut

200 OK · 201 Créé · 400 Mauvaise requête · 401 Non autorisé · 404 Introuvable · 429 Trop de requêtes · 500 Erreur serveur

Body

Données échangées - absent pour GET, présent pour POST/PUT (JSON)

Les verbes HTTP - CRUD sur une API

Verbe	Action CRUD	Idempotent ?	Exemple DVF	Code succès typique
GET	Lire	Oui	Récupérer les transactions d'une commune	200 OK
POST	Créer	Non	Soumettre une nouvelle transaction	201 Created
PUT	Remplacer	Oui	Remplacer une transaction complète	200 OK
PATCH	Modifier	Non	Modifier le prix d'une transaction	200 OK
DELETE	Supprimer	Oui	Supprimer une transaction	204 No Content

Dans ce cours, vous utiliserez quasi-exclusivement GET.

Les APIs de données publiques (data.gouv.fr, INSEE, geo.api.gouv.fr) exposent des endpoints en lecture seule - GET suffit pour 95 % des besoins en data engineering.

Codes de statut HTTP

2xx — Succès

200 OK Requête réussie — corps présent

201 Created Ressource créée (POST/PUT)

204 No Content Succès sans corps (DELETE)

4xx — Erreur client

400 Bad Request Requête malformée - vérifier les paramètres

401 Unauthorized Non authentifié - fournir un token

403 Forbidden Authentifié mais pas autorisé

404 Not Found Ressource inexistante

429 Too Many Requests Rate limit dépassé - attendre et retenter

5xx — Erreur serveur

500 Internal Error Bug côté serveur - réessayer plus tard

503 Unavailable Serveur surchargé ou en maintenance

Exercice live - Diagnostiquer ces réponses HTTP

Pour chaque réponse HTTP, identifiez : (1) s'il y a une erreur, (2) de quel côté (client ou serveur), (3) quoi faire.

Réponse A

HTTP/1.1 429 Too Many Requests

Retry-After: 60

X-RateLimit-Reset: 1700000060

Réponse B

HTTP/1.1 401 Unauthorized

WWW-Authenticate: Bearer realm='api'

Réponse C

HTTP/1.1 404 Not Found

Content-Type: application/json

```
{"error": "commune 99999 not found"}
```

À diagnostiquer :

Réponse A

Réponse B

Réponse C

Formats de réponse - JSON, XML, CSV

Les APIs peuvent retourner différents formats. Le client indique sa préférence via le header Accept. En data engineering, JSON est le standard dominant.

JSON - le standard universel

```
# Accept: application/json
[{"code": "59350", "nom": "Lille", "population": 233098},
{"code": "59512", "nom": "Roubaix", "population": 98978}]
```

JSON

Parsé nativement par Python (json.loads). Hiérarchique. Poids léger.
Standard des APIs modernes.

CSV

Simple pour les données tabulaires. Pas de types. Idéal pour exporter, pas pour des APIs complexes.

XML - legacy, encore présent

```
# Accept: application/xml
<communes>
  <commune code='59350'>
    <nom>Lille</nom><pop>233098</pop>
  </commune>
</communes>
```

XML

Verbeux, plus difficile à parser (lxml, xml.etree). Encore présent dans les APIs gouvernementales et bancaires.

Authentification - Pourquoi et comment s'identifier

La majorité des APIs exigent que vous prouviez votre identité avant de répondre. C'est la différence entre une API publique ouverte et une API professionnelle.

API Key

Le plus simple. Une clé secrète fournie par le service, passée dans le header ou l'URL.

→ *data.gouv.fr, OpenWeatherMap, Nominatim (optionnel)*

OAuth2 Client Credentials

Machine à machine. Votre application obtient un token temporaire avec son ID + secret.

→ *INSEE API Données, certaines APIs gouvernementales*

OAuth2 Authorization Code

L'utilisateur se connecte et autorise votre app. Flux web avec redirection.

→ *APIs réseaux sociaux, Google, GitHub*

Bearer Token / JWT

Un token signé qui prouve votre identité sans renvoyer votre mot de passe à chaque requête.

API Key - La méthode la plus courante

Une clé API est une chaîne secrète attribuée à votre compte. Elle identifie qui fait la requête et permet au service de contrôler vos quotas.

Bonne pratique - Header HTTP

```
import requests
from dotenv import load_dotenv
import os

load_dotenv() # charge le fichier .env
API_KEY = os.getenv('MY_API_KEY')

r = requests.get(
    'https://api.example.com/data',
    headers={'X-API-Key': API_KEY},
    timeout=10,
)
```

À éviter - Clé dans l'URL

```
# La clé apparaît dans les logs,
# l'historique navigateur, les URLs
# partagées par erreur...

r = requests.get(
    'https://api.example.com/data'
    '?api_key=sk-1234SECRET', #
)
```

Fichier .env (jamais dans Git)

```
MY_API_KEY=sk-votre-cle-secrete
```

Une clé API ne doit jamais apparaître dans le code source. Toujours utiliser une variable d'environnement ou un gestionnaire de secrets.

Discussion - Pourquoi ne jamais hardcoder une clé API

En 2023, plus de 10 millions de secrets ont été détectés dans des dépôts GitHub publics. Voici trois scénarios réels.

Scénario 1 — L'étudiant qui partage son code

Un étudiant pousse son projet sur GitHub public avec une clé OpenAI dans le code. Des bots scannent GitHub en continu. En moins de 5 minutes, la clé est détectée, utilisée pour générer du texte, et la facture atteint 300 \$ avant qu'il s'en rende compte.

Scénario 2 — Le data engineer et les logs

Un pipeline log toutes ses requêtes pour le débogage. L'URL contient `?api_key=secret`. Les logs sont stockés dans un système de monitoring partagé avec toute l'équipe y compris des prestataires externes.

Scénario 3 — Le Git rewrite

Un développeur réalise qu'il a commité une clé. Il la supprime dans un nouveau commit. La clé reste visible dans l'historique Git accessible avec `git log` sur tout clone existant.

OAuth2 Client Credentials - Machine à machine

Ce flux est utilisé quand votre script (un serveur, un pipeline) doit accéder à une API sans intervention humaine. Vous échangez un ID + secret contre un token temporaire.

1. Votre app



envoie client_id + client_secret

2. Serveur
OAuth



vérifie et génère un access_token
(ex: 1h)

3. Votre app



utilise le token dans chaque requête
API

4. Token
expiré



→ recommencer l'échange (étapes
1-2)

```
import requests

# Étape 1 : obtenir un token
token_resp = requests.post('https://auth.example.com/token', data={
    'grant_type' : 'client_credentials',
    'client_id' : os.getenv('CLIENT_ID'),
    'client_secret': os.getenv('CLIENT_SECRET'),
})
access_token = token_resp.json()['access_token']
expires_in = token_resp.json()['expires_in'] # secondes

# Étape 2 : appeler l'API avec le token
r = requests.get('https://api.example.com/data',
    headers={'Authorization': f'Bearer {access_token}'})
)
```

Identifier le mode d'authentification

Pour chacune de ces APIs que vous utiliserez en TP, lisez les indices et identifiez le mécanisme d'authentification.

geo.api.gouv.fr

→ ???

Pas de clé requise. Les requêtes sont publiques.

data.gouv.fr

→ ???

Documentation : 'Générez votre clé depuis votre profil.' Paramètre X-API-KEY dans les headers.

API INSEE Données

→ ???

Documentation : 'Inscription sur api.insee.fr, obtenez Consumer Key + Secret.'

OpenWeatherMap

→ ???

Documentation : 'Ajoutez &appid=VOTRE_CLE à chaque URL.'

Du code fragile au client robuste

Un appel API qui fonctionne une fois ne suffit pas. En production, les réseaux sont instables, les serveurs saturent, les quotas se dépassent. Votre code doit anticiper tout ça.

Code fragile - 1 ligne

```
r = requests.get(url)
data = r.json()
# Plante si réseau coupé
# Plante si 429 ou 500
# Plante si JSON invalide
# Plante si timeout
```

Timeout

Sans timeout, une requête peut bloquer votre script indéfiniment si le serveur ne répond pas.

Retry

Les erreurs 429 (rate limit) et 5xx (serveur) sont temporaires. Une logique de retry évite l'échec sur des problèmes passagers.

✓ Client robuste - pattern complet

```
r = requests.get(
    url,
    headers=headers,
    timeout=10,          # timeout
    params=params,
)
r.raise_for_status()    # lève si 4xx/5xx
data = r.json()
```

raise_for_status()

Lève une exception immédiatement sur tout code 4xx ou 5xx. Sans ça, `r.json()` peut retourner un message d'erreur que vous prenez pour des données.

Logging

Tracer chaque appel (URL, statut, temps) est indispensable pour déboguer un pipeline en production.

Retry avec backoff exponentiel

Un serveur qui répond 429 ou 503 vous dit 'je suis surchargé'. La bonne réponse n'est pas d'insister immédiatement - c'est d'attendre de plus en plus longtemps.

```
import requests, time, random
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def creer_session(retries=3, backoff=1.0) -> requests.Session:
    """Session avec retry automatique sur 429, 500, 502, 503, 504."""
    session = requests.Session()
    retry = Retry(
        total = retries,
        backoff_factor = backoff, # attente : 1s, 2s, 4s, 8s...
        status_forcelist = [429, 500, 502, 503, 504],
        allowed_methods = ['GET', 'POST'],
    )
    adapter = HTTPAdapter(max_retries=retry)
    session.mount('https://', adapter)
    session.mount('http://', adapter)
    return session

# Utilisation
session = creer_session()
r = session.get(url, headers=headers, timeout=10)
r.raise_for_status()
```

Tentative 1

Échec 503 → attendre 1 s

Tentative 2

Échec 503 → attendre 2 s

Tentative 3

Échec 503 → attendre 4 s

Tentative 4

✓ 200 OK → continuer

Rate Limiting - Respecter les quotas

Toutes les APIs limitent le nombre de requêtes autorisées par unité de temps. Dépasser ce seuil déclenche une erreur 429. La clé : lire les headers de réponse.

```
def appel_respectueux(session, url, headers, delai_min=0.2):
    """Appel API avec lecture du rate limit et pause adaptative."""
    r = session.get(url, headers=headers, timeout=10)

    # Lire les headers de rate limit (varient selon l'API)
    restant = int(r.headers.get('X-RateLimit-Remaining', 999))
    reset_ts = int(r.headers.get('X-RateLimit-Reset', 0))

    if r.status_code == 429:
        retry_after = int(r.headers.get('Retry-After', 60))
        print(f'Rate limit atteint. Pause de {retry_after}s...')
        time.sleep(retry_after)
        return appel_respectueux(session, url, headers) # retry

    # Ralentir proactivement si quota presque épuisé
    if restant < 10:
        time.sleep(1.0) # pause longue
    else:
        time.sleep(delai_min) # pause minimale entre requêtes

    r.raise_for_status()
    return r.json()
```

X-RateLimit-Remaining

Combien de requêtes il reste dans la fenêtre courante. Ralentir quand il approche de 0.

X-RateLimit-Reset

Timestamp UNIX de remise à zéro du compteur. Calculez le temps à attendre.

Retry-After

Sur une 429, combien de secondes attendre avant de retenter. À respecter absolument.

Pause minimale

Même sans 429, toujours ajouter un délai minimal (100-500 ms) entre requêtes. C'est du civisme.

Caching des réponses - requests-cache

Certaines données changent rarement (noms de communes, codes INSEE). Les redemander à chaque exécution est inutile, lent, et épuise le quota. Le cache résout ça.

```
import requests_cache

# Activer le cache – SQLite local, TTL de 24h
requests_cache.install_cache(
    'api_cache',
    backend='sqlite',
    expire_after=86_400, # 24 heures en secondes
)

# Identique à requests, transparent pour votre code !
import requests
r = requests.get('https://geo.api.gouv.fr/departements/59/communes')

# 1ère exécution : appel réseau → stocké en cache
# Exécutions suivantes : réponse instantanée depuis le cache
print('Depuis le cache :', r.from_cache if hasattr(r, 'from_cache') else
      'non')

# Forcer une mise à jour
with requests_cache.disabled():
    r_fresh = requests.get(url)
```

TTL (Time To Live)

Durée de validité d'une entrée en cache. Après expiration, le prochain appel refait une vraie requête.

Clé de cache

Construite à partir de l'URL + paramètres. Deux requêtes identiques partagent la même entrée.

Cache invalidation

`requests_cache.clear()` vide tout.
`requests_cache.disabled()` désactive temporairement.

Quand l'utiliser ?

Données de référence (communes, codes), développement (économise les quotas), tests (reproductibilité).

requests.Session vs appels unitaires

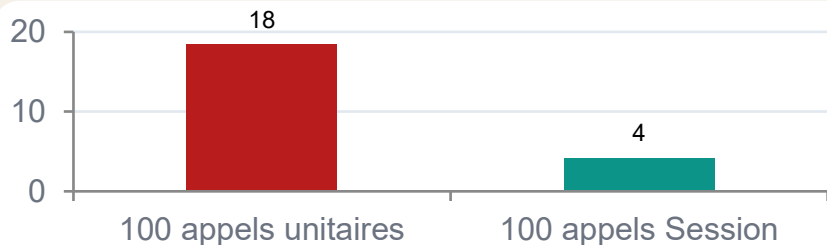
Chaque appel `requests.get()` ouvre et ferme une connexion TCP. Sur 1000 requêtes, c'est 1000 handshakes inutiles. Session réutilise la même connexion.

Appels unitaires - inefficace

```
# 1000 appels = 1000 connexions # + 1000 headers à répéter
for code in codes_communes:
    r = requests.get(
        f'{BASE_URL}/{code}',
        headers={'X-Api-Key': KEY}, # répété
    )
```

Session - connexion persistante

```
# 1 connexion réutilisée
# Headers définis une seule fois
with requests.Session() as session:
    session.headers['X-Api-Key'] = KEY
    for code in codes_communes:
        r = session.get(
            f'{BASE_URL}/{code}'
        )
```



Session est 4x plus rapide sur 100 requêtes - 18x sur 1000.

Pourquoi valider les réponses API ?

Une API peut changer son schéma sans préavis. Un champ devient optionnel, un type change, un champ est renommé. Sans validation, votre pipeline plante silencieusement ou pire, continue avec des données incorrectes.

Sans validation - crash silencieux

```
data = requests.get(url).json()

# L'API a renommé 'population' en 'pop'
# Ici : data['population'] -> KeyError !
# Ou pire : data['price'] est maintenant
# une string '2 500' au lieu d'un int...
# Votre calcul continue avec des NaN.
```

✓ Avec Pydantic - erreur explicite

```
from pydantic import BaseModel

class Commune(BaseModel):
    code: str
    nom: str
    population: int

data = requests.get(url).json()
communes = [Commune(**c) for c in data]
# Si 'population' manque -> ValidationError
# immédiate, claire, avec le champ fautif
```

Pydantic transforme vos données en objets Python typés, validés à la création. Un schéma qui change = une erreur immédiate et explicite, pas un bug subtil 3 heures plus tard.

Pydantic - Modèles, types et champs optionnels

```
from pydantic import BaseModel, Field, field_validator
from typing import Optional
from datetime import date

class Transaction(BaseModel):
    id_mutation: str
    date_mutation: date
    valeur_fonciere: float = Field(gt=0, lt=10_000_000)
    type_local: str
    surface_bati: Optional[float] = None # champ optionnel
    nom_commune: str

    # Valideur custom : la date ne peut pas être dans le futur
    @field_validator('date_mutation')
    @classmethod
    def date_pas_future(cls, v):
        if v > date.today():
            raise ValueError(f'Date dans le futur : {v}')
        return v

# Désérialisation et validation en une ligne
t = Transaction(**api_response)

# Accès typé (autocomplétion IDE !)
print(t.valeur_fonciere) # float garanti
print(t.date_mutation.year) # date, pas str
```

BaseModel

Classe de base Pydantic. Héritez-en pour créer un modèle. La validation est automatique à l'instanciation.

Field(gt=0)

Contraintes supplémentaires : gt (greater than), lt, ge, le, min_length, max_length, regex...

Optional[T]

Champ qui peut être None. Pydantic accepte l'absence ou la valeur null dans le JSON.

field_validator

Logique de validation custom. Lève ValueError si la valeur est invalide. Appelé après la conversion de type.

Modéliser cette réponse JSON avec Pydantic

Voici la réponse réelle de l'API geo.api.gouv.fr. Écrivez le modèle Pydantic correspondant en complétant la classe.

```
# Réponse API : GET
/communes?codeDepartement=59&fields=nom,code,population,co
desPostaux
[
  {
    "nom": "Lille",
    "code": "59350",
    "population": 233098,
    "codesPostaux": ["59000", "59800", "59260"]
  },
  {
    "nom": "Sains-du-Nord",
    "code": "59543",
    "population": null,    # peut être null !
    "codesPostaux": ["59177"]
  }
]
```

```
from pydantic import BaseModel
from typing import ???

class Commune(BaseModel):
    nom:        ???
    code:       ???
    population: ???
    codesPostaux: ???

# Instanciation
data = requests.get(url).json()
communes = [Commune(*c) for c in data]

# Test
print(communes[0].nom)
print(communes[1].population)
```

Tour des APIs publiques françaises essentielles

API	Base URL	Auth	Données	Quota
geo.api.gouv.fr	geo.api.gouv.fr	Aucune	Communes, depts, régions, géométrie	Généreuse
data.gouv.fr	www.data.gouv.fr/api/1	Optionnelle (API Key)	Datasets, organisations, ressources	Généreuse
API INSEE	api.insee.fr	OAuth2 Client Cred.	Séries temporelles, census, pop.	200 req/min
BAN (adresses)	api-adresse.data.gouv.fr	Aucune	Géocodage, autocomplétion adresses	50 req/s
Nominatim OSM	nominatim.openstreetmap.org	Aucune (User-Agent requis)	Géocodage, reverse-geocoding	1 req/s
DVF open data	api.cquest.org/dvf	Aucune	Transactions immobilières	Raisonnabe

Pagination - Récupérer tous les résultats

Aucune API ne retourne 100 000 résultats d'un coup. Elles paginent. Votre code doit itérer sur toutes les pages pour récupérer l'ensemble des données.

Pagination par offset (data.gouv.fr, la plus courante)

```
def paginer_tout(session, url, params={}, page_size=100):
    """Récupère tous les résultats en paginant automatiquement."""
    resultats, page = [], 0
    while True:
        r = session.get(url, params={**params,
                                     'page': page, 'page_size': page_size}, timeout=10)
        r.raise_for_status()
        data = r.json()
        items = data.get('data', data) # selon le format
        if not items: break             # plus de résultats
        resultats.extend(items)
        page += 1
        print(f'Page {page} - {len(resultats)} résultats cumulés')
    return resultats
```

Pagination par curseur

```
# next_cursor dans la réponse
cursor = None
while True:
    params = {'limit':100}
    if cursor:
        params['cursor'] = cursor
    data = session.get(
        url, params=params).json()
    items = data['results']
    if not items: break
    resultats.extend(items)
    cursor = data.get('next')
    if not cursor: break
```

API ou fichier statique : quand choisir quoi ?

Les données DVF sont disponibles à la fois en CSV (data.gouv.fr) et via une API. Comment choisir ?

Critère	Fichier statique (CSV/Parquet)	API REST
Fraîcheur	Snapshot à un instant T	Données en temps réel ou quasi
Volume	Tout d'un coup, même si vous n'avez besoin que d'une ligne	Filtrage côté serveur : ne récupérer que ce dont on a besoin
Reproductibilité	✅ Le fichier ne change pas	⚠️ Les données peuvent changer entre deux exécutions
Débit	Téléchargement unique, très rapide	1 requête / résultat → lent sur grand volume
Complexité	Simple : <code>read_csv()</code>	Requiert auth, retry, pagination, validation
Cas d'usage	Analyse batch, exploration, ML	Enrichissement en temps réel, alertes, intégrations

MCP - Model Context Protocol : le standard d'Anthropic

Les LLMs comme Claude sont puissants mais isolés. MCP est le protocole ouvert créé par Anthropic pour connecter standardisé des modèles d'IA à des outils, des APIs et des sources de données.

Problème

Chaque application IA devait créer son propre système d'intégration - des dizaines de connecteurs ad hoc incompatibles entre eux et difficiles à maintenir.

Solution MCP

Un protocole universel : n'importe quel modèle peut parler à n'importe quel outil via MCP, comme HTTP a standardisé la communication entre navigateurs et serveurs web.

Analogie

MCP est aux LLMs ce que HTTP est au web.
Un standard ouvert, simple, que tout le monde peut implémenter.

Architecture MCP - Host, Client, Server

MCP définit 3 rôles distincts qui communiquent selon un protocole standardisé. Chaque rôle a une responsabilité précise.



Les 3 primitives qu'un MCP Server expose :

Tools

Fonctions que le modèle peut appeler. Équivalent des endpoints POST d'une API. Ex: `search_web()`, `execute_sql()`, `geocode_address()`

Resources

Données exposées en lecture. Équivalent des endpoints GET. Ex: `file://projet/data.csv`, `db://table/users`

Prompts

Templates de prompts réutilisables fournis par le serveur au modèle. Ex: 'Analyse ce dataset DVF selon...'

MCP et REST - Des niveaux d'abstraction différents

MCP ne remplace pas les APIs REST — il les enveloppe. Un MCP Server est souvent une fine couche au-dessus d'une API REST existante.

	API REST	MCP Server
Consommé par	Code Python, navigateur, mobile	Modèle LLM (Claude, GPT, Mistral...)
Interface	HTTP (GET, POST, JSON)	JSON-RPC 2.0 sur stdio ou HTTP/SSE
Découverte	Documentation Swagger/OpenAPI	list_tools(), list_resources() automatiques
Auth	API Key, OAuth2, Bearer	Héritée du serveur sous-jacent
Cas d'usage	Application → données	LLM → outil → données

Exemple — `geo.api.gouv.fr` enveloppé dans un MCP Server :

Avant — le data engineer code ça :

```
r = session.get('https://geo.api.gouv.fr/'
               'departements/59/communes',
               params={'fields': 'nom,code,population'})
communes = r.json()
```

Après — Claude appelle le MCP Server :

```
# Claude demande (en langage naturel) :
# 'Liste les communes du Nord avec leur pop'
# Le MCP Server fait l'appel API
# et retourne le résultat formaté
```

Écosystème MCP - Ce qui existe déjà

Depuis son lancement par Anthropic en novembre 2024, MCP a été adopté rapidement. Des centaines de serveurs MCP publics sont disponibles.

MCP Server	Expose	Utile pour un data engineer
Filesystem	Lire/écrire des fichiers locaux	Charger un CSV, écrire des résultats
PostgreSQL / SQLite	Requêtes SQL en langage naturel	Explorer un schéma, analyser des données
GitHub	Issues, PRs, code, commits	Review de code, génération de PR
Google Drive	Docs, Sheets, fichiers	Lire des rapports, mettre à jour des tableaux
Web Search	Recherche web en temps réel	Enrichir des analyses avec des données récentes
Fetch / HTTP	Appeler n'importe quelle URL	Wrapper de secours pour toute API REST
Jupyter / Notebook	Exécuter des cellules Python	IA qui code et exécute directement
Slack	Messages, canaux, notifications	Alertes automatiques sur anomalies données

Checklist du client API robuste en production

01

Jamais de secret dans le code

Variables d'environnement (.env) ou gestionnaire de secrets (Vault, AWS Secrets Manager). .env dans .gitignore.

02

Toujours un timeout

`requests.get(url, timeout=10)`. Sans timeout, une requête peut bloquer indéfiniment.

03

Toujours `raise_for_status()`

Lever une exception explicite sur toute réponse 4xx/5xx. Ne jamais ignorer les codes d'erreur.

04

Retry sur les erreurs temporaires

`urllib3.Retry` sur 429, 500, 502, 503, 504 avec backoff exponentiel. Pas de retry sur 400, 401, 404.

05

Cache pour les données stables

`requests-cache` ou cache fichier JSON pour les référentiels (communes, codes INSEE) qui ne changent pas.

06

Valider le schéma des réponses

`Pydantic` pour détecter les ruptures de contrat API avant qu'elles corrompent votre pipeline.

Discussion finale

Avant de passer au TP, trois questions pour consolider les concepts.

Question 1

Votre pipeline de géocodage DVF s'exécute chaque nuit sur 500 nouvelles transactions. Il échoue un matin en silence parce que Nominatim a changé son format de réponse : le champ 'lat' s'appelle maintenant 'latitude'. Votre DataFrame contient 500 lignes de NaN. Qu'est-ce qui aurait pu empêcher ça ?

Question 2

Vous devez géocoder 50 000 adresses DVF pour une livraison dans 8 heures. Nominatim est limité à 1 req/s, donc 50 000 secondes = 13h. Vous n'avez pas accès à une API payante. Que faites-vous ?

Question 3

Un collègue propose de mettre la clé API INSEE directement dans le fichier de configuration YAML versionné dans Git, 'pour que tout le monde puisse lancer le pipeline facilement'. Comment lui répondez-vous ?

Questions ?

ahmad.almosaalmaksour@lacatholille.fr